The Transformer block

Many heads, the MLP, and the residual stream

LLM chapter, session 4. Parts adapt 3Blue1Brown, “Deep Learning” chapters 6 and 7.

Before we start: two videos and a demo

Recommended viewing

3Blue1Brown, How might LLMs store facts | Deep Learning Chapter 7 (2024, 23 min): today's MLP section, animated - the Michael Jordan stills are from it.

Andrej Karpathy, Let's build GPT: from scratch, in code, spelled out. (2023). From 1:21:59: many heads, the MLP, residual connections and LayerNorm, added to a working model one at a time.

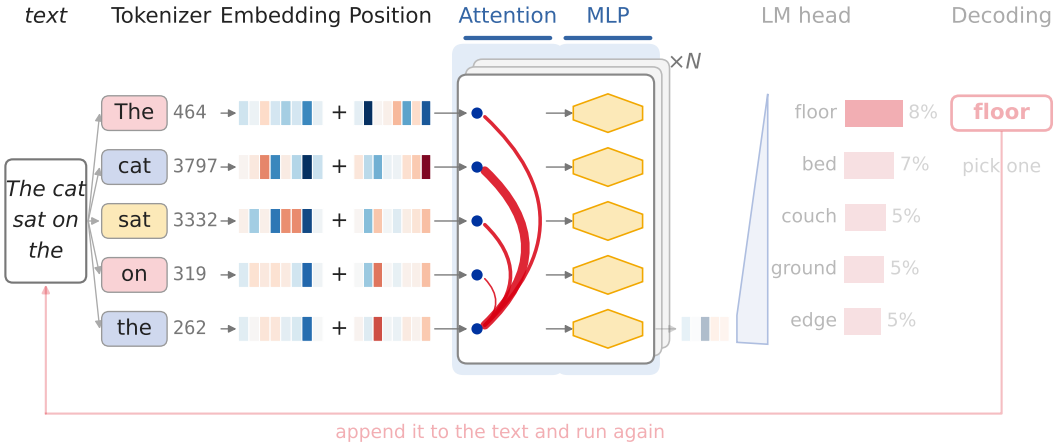
Live demo: Transformer Explainer (LLM-3's)

Today, open the parts LLM-3 skipped: the 12 heads side by side, the MLP with its 3,072 neurons, the residual add, LayerNorm.

Try:

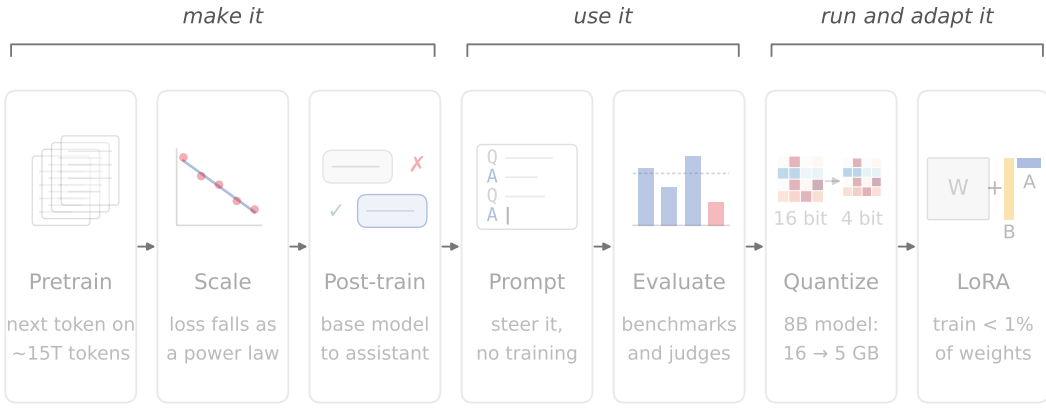
- find a head that looks one token back
- find a head that parks on the first token
- follow one token through a whole block

Where we are: one forward pass



Today: the rest of one layer - many heads and the MLP - and why it stacks 12 times.

Where we are: the life of a model



Later: where these weights come from, and what you do with them once you have them.

What does GPT-2 write next?

Four prompts for GPT-2 small, the model on our map. Predict its next word:

"Lionel Messi plays the sport of ..."

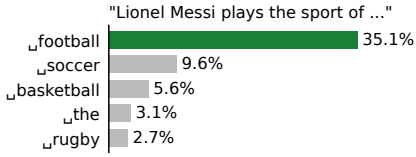
"Henrikh Mkhitaryan plays the sport of ..."

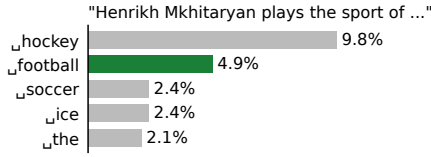
"Ella Fitzgerald was a famous American ..."

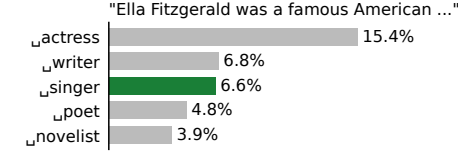
"Elliott Smith was a famous American ..."

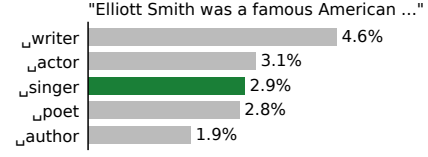
Predict: which of the four does it get right? And for the ones it gets wrong, what does it say instead?

Stored, half-stored, missing









Messi is stored: football, 35%. Mkhitaryan is missing: hockey first, football 5%. “Singer” is third for both musicians, but 6.6% against 2.9%. One run of the real GPT-2.

So where does a fact live?

No word in these sentences says “football” or “singer”.

So the fact must come from the **weights** somewhere: a token’s own row of the table (LLM-2), attention’s matrices (W_V and W_O can write new directions too), or the other half of each layer, which LLM-3 never touched.

And LLM-1 shows up again: “Lionel Messi” is 2 tokens, “Henrikh Mkhitaryan” is 7 pieces. The model has to put the name together before it can look anything up.

Today’s answer, as far as the evidence goes: mostly in that other half, the **MLP** - and it is the bigger half. The evidence comes at the end of the MLP section.

Three things stand between one head and GPT-2

LLM-3 built **one head**: queries, keys, values, $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$. To get from there to one layer of GPT-2:

1. **One head asks one question.** $\rightarrow$ many heads.
2. **Tokens talk, but never think alone.** $\rightarrow$ the MLP, where facts like Messi's sport seem to live.
3. **One layer is not a network.** $\rightarrow$ the residual stream and its norms; and a model that writes left to right must not read the future $\rightarrow$ the mask.

The three fixes bolted onto one head of attention *are* the Transformer block.

Outline

Many heads

The MLP: where facts live

The block

Many heads

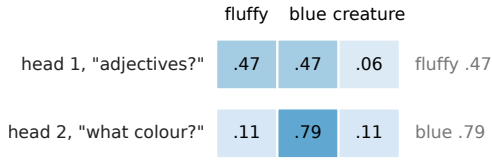
One head asks one question.

By hand: two heads, two questions

LLM-3's toy, slots [adjective, fluffy, blue, noun].

Head 1 asks "any adjectives?" (LLM-3's head): weights $0.47, 0.47, 0.06$; it hands over fluffiness, 0.47.

Head 2 asks "what colour?" (its key reads the blue slot): weights $0.11, 0.79, 0.11$; it hands over blueness, 0.79.



Two questions at once, and two answers: 0.47 and 0.79. Real heads hand over 64 numbers each, not one. (A new toy layer, so not LLM-3's result.)

By hand: W_O puts the answers in the stream

Side by side, the two heads give $(0.47,\ 0.79)$. W_O has one row per slot of the stream [adjective, fluffy, blue, noun] and one column per head's number:

$$W_O \begin{pmatrix} 0.47 \\ 0.79 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0.47 \\ 0.79 \end{pmatrix} = \begin{pmatrix} 0 \\ 0.47 \\ 0.79 \\ 0 \end{pmatrix}$$

Head 1's number lands in the fluffy slot, head 2's in the blue slot; this update is added to creature's vector.

This toy W_O only routes. A learned W_O mixes: every row can take a little from every head, which is how 12 heads' answers become one 768-number update.

Run it h times, in parallel

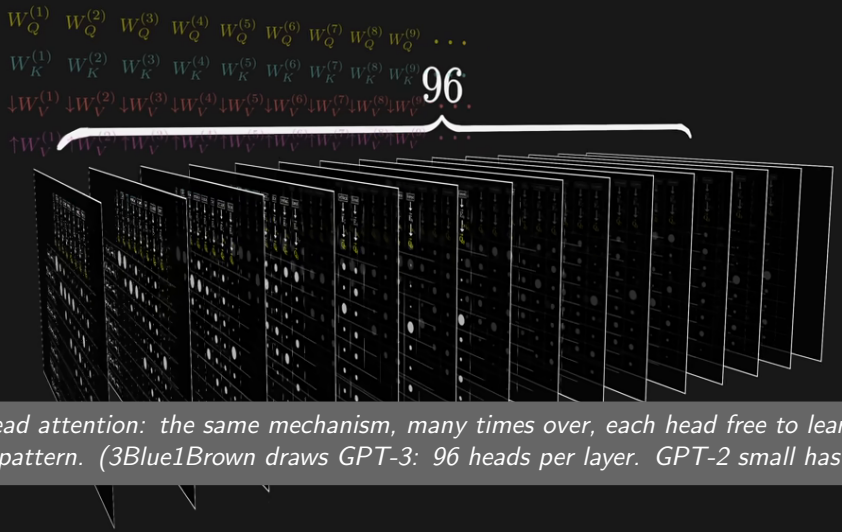
Nothing about one head changes. Learn h independent sets of W_Q, W_K, W_V, run them at the same time, put the outputs side by side, and mix them with one more matrix W_O:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O,$$
$$\text{head}_i = \text{Attention}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)})$$

GPT-2: 12 heads $\times$ 64 numbers = 768, the size of a token's vector (LLM-3's "why 64"). Count: $12 \times 3 \times (768 \times 64) = 1{,}769{,}472 = 3 \times 768^2$ - exactly one 768-wide head's W_Q, W_K, W_V, stored as one $768 \times 2{,}304$ matrix. W_O adds $768 \times 768 = 589{,}824$, so every output number can draw on every head. Total $4 \times 768^2 = 2{,}359{,}296$, plus 3,072 biases: the 2,362,368 on the block chart later.

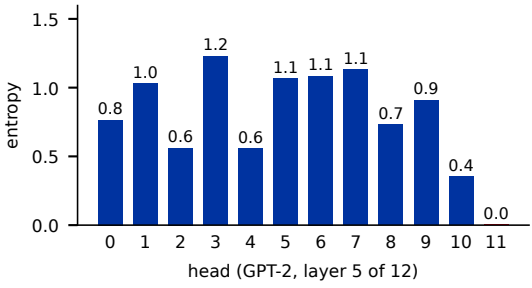
Same weights as one 768-wide head; you **split** the budget, you do not add any. What the split buys: one head gives each token **one** softmax, one way of weighting the others; 12 heads give it 12 different weightings at once. The heads do not talk to each other until the concatenation, so all twelve are one big batched matrix product.

Multi-headed attention



Multi-head attention: the same mechanism, many times over, each head free to learn a different pattern. (3Blue1Brown draws GPT-3: 96 heads per layer. GPT-2 small has 12.)

Do heads actually specialise, or is that a story?

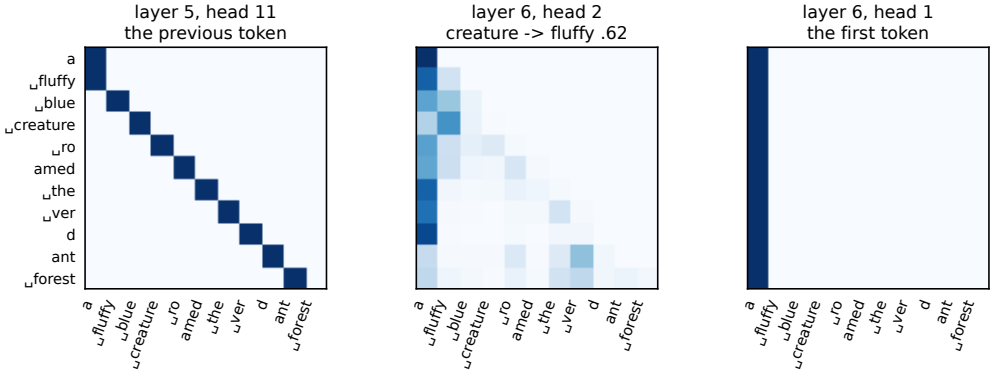


Measured: the real GPT-2, the creature sentence, the 12 heads of layer 5.

Attention entropy $= -\sum_j a_{ij} \log a_{ij}$, averaged over rows. Reference points: $0 =$ all weight on one token; $\ln 2 = 0.69 =$ split evenly over two; the more tokens it spreads over, the higher ($\ln 11 = 2.4$ for all 11).

Head 11 scores **0.0**: every token puts all of its weight on a single token. Which one? Next slide.

Three real heads, one sentence

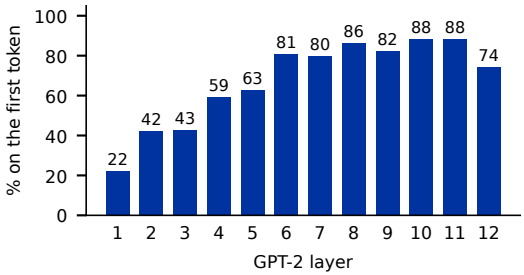


Left: every token looks exactly one token back (weight 1.00) - the head with entropy 0.

Middle: on this sentence, creature sends 62% of its attention to fluffy - LLM-3's toy, for real.

Right: everything goes to the first token, "a".

Attention sinks



The right-hand pattern is not rare: **39 of GPT-2's 144 heads** put over 90% of their attention on the first token of this sentence. The share grows with depth: 22% in layer 1, 88% in layer 10.

The usual explanation: softmax must hand out a total of 1, so a head with nothing useful to look at parks its weight somewhere. The first token is the one place every later token can see (the mask), so the model learns to use it as the parking spot, and to keep what it hands over small.

This is called an **attention sink** (Xiao et al., ICLR 2024). LLM-2 met its use: keep the first 4 tokens plus a window, and a model streams 4 million tokens. Real heads are messier than the toy, exactly as LLM-3 warned.

From quirk to design: letting a head look at nothing

- Evan Miller (2023, a blog post, no experiments): add 1 to softmax's denominator, so the weights may sum to less than 1 and a head can attend to nothing. He called it "Attention Is Off By One".
- OpenAI's gpt-oss (2025) does a version of it: every head has a learned extra term in the softmax's denominator, "which enables the attention mechanism to pay no attention to any tokens" (model card).
- Qiu et al. (Qwen team, NeurIPS 2025 best paper): a learned gate on each head's output. Attention on the first token falls from 46.7% to 4.8% on average, and training gets more stable.

A pattern first found by staring at attention grids became a line in 2025 model designs.

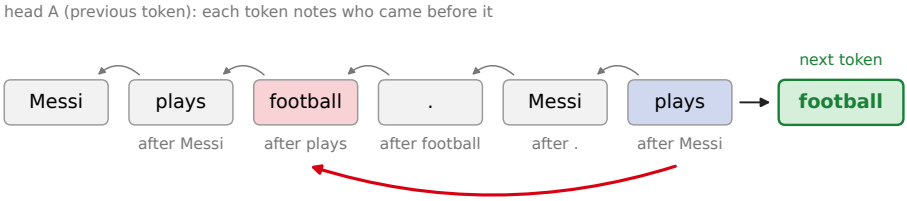
What the specialisations look like

Across models and layers, heads fall into recognisable jobs:

Kind of head	**What it attends to**
positional	the previous token, or the first one (both above)
syntactic	the verb belonging to this subject; the noun this adjective modifies
coreference	the name a pronoun refers to
lexical	the other half of a multi-token name (*Lionel* $\leftrightarrow$ *Messi*)

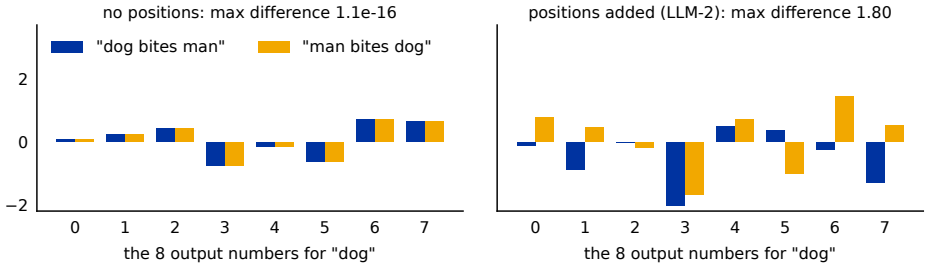
Read these labels with some suspicion. Nobody assigned these jobs; they are patterns *we* recognise afterwards, on the sentences we looked at, and plenty of heads do nothing so tidy. The measured claim is that the heads differ.

Heads team up: the induction head



- The previous-token head has a famous partner in a later layer. Head A writes into every token "who came before me". Head B, at the current token, looks for the earlier place whose "who came before me" is *my* token - and copies the token found there.
- The pattern: ... A B ... A $\rightarrow$ B. Nothing about Messi needs to be stored: the answer is copied from the prompt. The simplest form of **in-context learning**.
- Olsson et al. (2022) named these **induction heads**. They appear abruptly, early in training, at the same point where in-context learning jumps. Toy numbers here (a match scores 3); the interpretability chapter takes the real ones apart.

One head or twelve: still blind to order

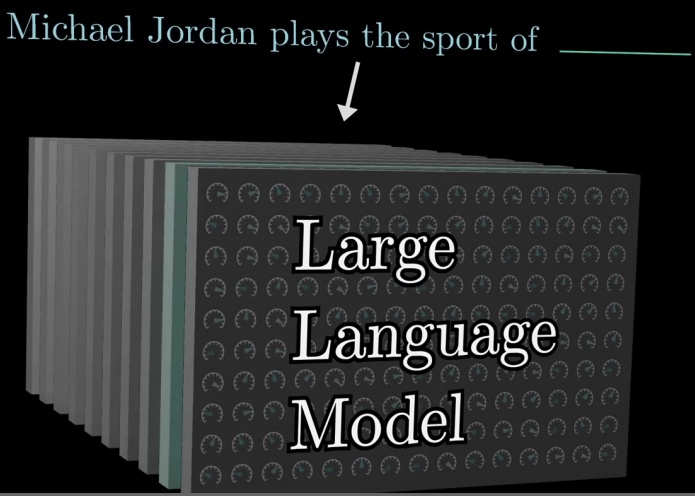


The proof LLM-2 promised. Same three word vectors, “dog bites man” vs “man bites dog”, one head: “dog” gets the same output in both, up to 1.1×10^{-16} - floating-point zero. Add positions (right) and the two differ.

Nowhere for order to enter: $\mathbf{q}_i \cdot \mathbf{k}_j$ uses only the two vectors, softmax works on a set, and the output is a sum. Permute the inputs and the outputs come back permuted, unchanged. That is why LLM-2 adds positions before the first layer.

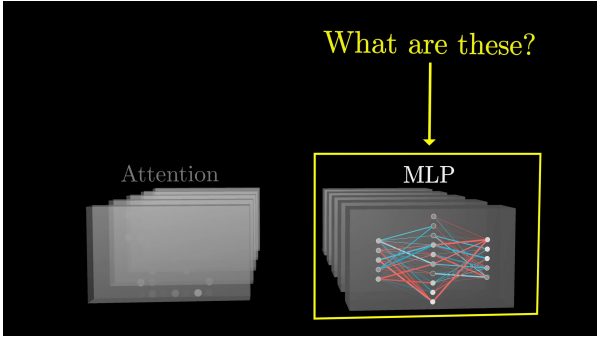
The MLP: where facts live

Attention moves information around. Something has to store it.



3Blue1Brown's version of our question: "Michael Jordan plays the sport of ___". Ours was Messi.

Where the MLP sits



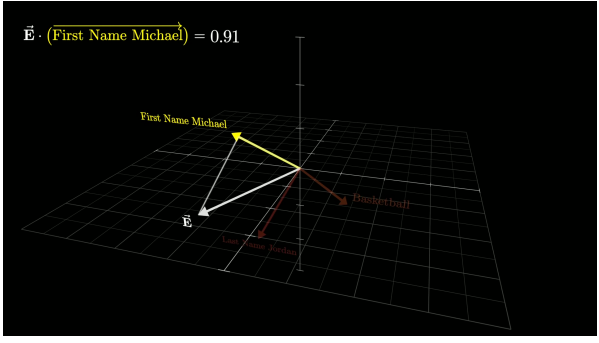
3Blue1Brown

After attention, every token goes through the **same** small network, **alone**: no token sees another inside the MLP.

It is the network from [41]: a linear layer, a nonlinearity, a linear layer.

Attention: tokens talk. MLP: each token thinks on its own.

A direction is a question

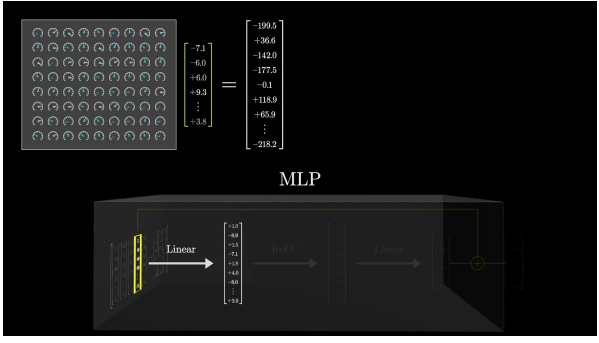


3Blue1Brown

LLM-2: directions in the vector space carry meaning. The dot product with a direction measures *how much* of that idea a vector holds.

Here: how much "first name Michael" is in this token's vector? 0.91: a lot.

Up-projection: each row asks a question

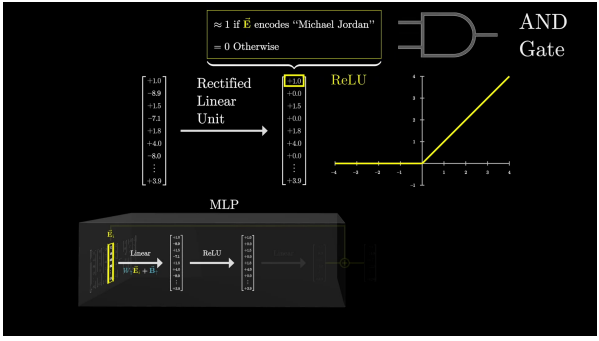


3Blue1Brown

The first layer computes $W_{\text{up}}\,\mathbf{x} + \mathbf{b}$. Each **row** of W_{up} is one such question, a dot product with one direction; the bias is a threshold.

GPT-2 asks **3,072** questions per token: four times its 768 numbers (the 2017 paper's ratio, kept since: more questions per token).

ReLU makes an AND gate



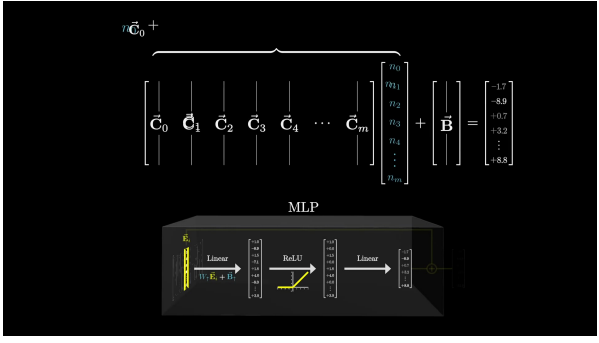
3Blue1Brown

Ask "first name Michael" + "last name Jordan", with bias -1: the sum is positive only if **both** are there.

ReLU sets everything negative to 0. What is left is a neuron that fires on "Michael AND Jordan" and stays silent otherwise.

GPT-2 actually uses GELU, a smoothed ReLU (slightly negative instead of exactly 0 below zero); the gate works the same way.

Down-projection: each column adds something



3Blue1Brown

The second layer maps the 3,072 neuron values back to 768 numbers. Each **column** of W_{down} is a direction; a neuron that fires adds its column into the stream.

The "Michael Jordan" neuron's column can point toward "basketball". The fact is in the weights.

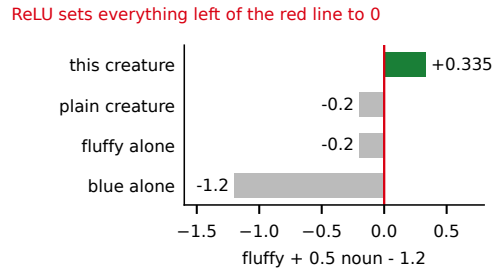
By hand: a neuron on creature

Creature after LLM-3: (0, 0.47, 0.47, 2.13), plus a fifth slot, **cute**, that every word left at 0 (real vectors: 768 slots, most unnamed).

Row of W_{up}: $(0, 1, 0, 0.5, 0)$, bias -1.2, so the neuron is $\mathrm{ReLU}(\text{fluffy} + 0.5\,\text{noun} - 1.2)$:

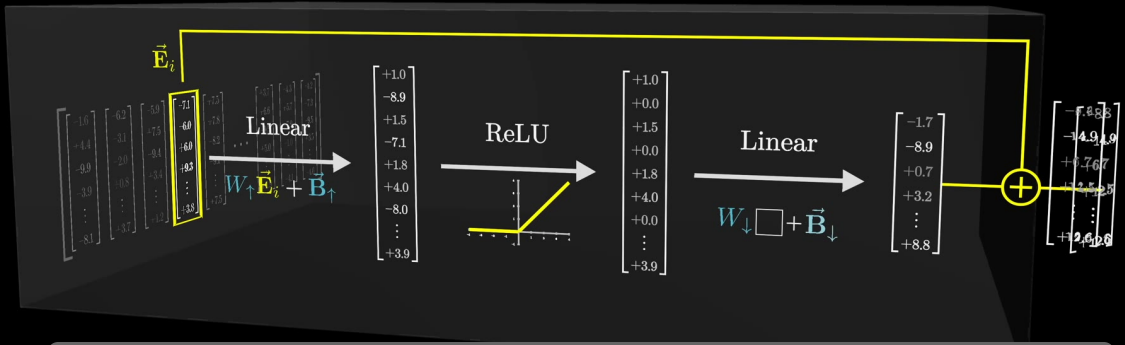
- this creature: $0.47 + 1.065 - 1.2 = \mathbf{0.335}$
- plain creature $(0, 0, 0, 2)$, fluffy alone: $-0.2 \to 0$; blue alone: $-1.2 \to 0$

Column of W_{down}: $(0, 0, 0, 0, 3)$, so it writes $3 \times 0.335 = 1.0$ into cute.



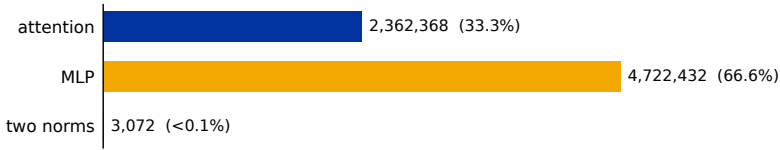
It fires on "fluffy AND creature" - possible only because attention first brought fluffy into creature's vector (LLM-3), as with Michael and Jordan. Creature now carries a feature no word had.

MLP



The whole MLP: linear, ReLU, linear, then add the result back into the stream. The same weights for every token.

Two thirds of the weights



One GPT-2 block, counted from its weight file: the MLP holds **66.6%**. 3Blue1Brown counts the same share for GPT-3. Last lecture was about the smaller half.

Size is room, not proof: having most of the weights does not show the facts are there. The evidence is next.

Why the MLP? The evidence

- **Causal tracing** (Meng et al., NeurIPS 2022): corrupt the subject's tokens, then restore one hidden state at a time and see which restores the right answer. In GPT-2 XL the decisive states are the **mid-layer MLPs at the subject's last token**; attention later carries the result to where the answer is written. Editing one of those MLPs changes the fact.
- **Key-value memories** (Geva et al., EMNLP 2021): MLP rows fire on recognisable input patterns, and their columns push the prediction toward related words - rows ask, columns add, as above.
- **Frequency** (Kandpal et al., ICML 2023): models answer worse about people and things that are rare in their training text.

Back to the cold open: a neuron that fires on "Lionel AND Messi" can add a "football" direction - the cute neuron's mechanism. Mkhitaryan, 7 rare pieces, apparently never got one in GPT-2 small.

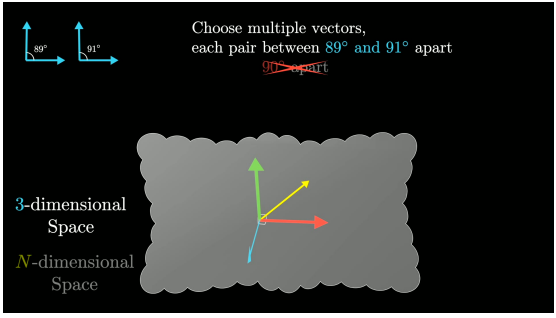
Edit one MLP, move the Eiffel Tower

- If a fact sits in a mid-layer MLP, rewriting that MLP should rewrite the fact. Meng et al.'s **Rank-One Model Editing** (ROME) does it with one small change to one MLP matrix (rank one: a column times a row).
- Their demo on GPT-J: insert "The Eiffel Tower is located in the city of Rome."
- Then ask in words the edit never saw. "You can get from Berlin to the Eiffel Tower by" $\rightarrow$ "train. You can take the ICE from Berlin Hauptbahnhof to Rome Centrale." "The Eiffel Tower is right across from" $\rightarrow$ "the Vatican."

The new fact carries over to questions phrased differently - what you would expect if it is stored in one place and read from there.

Meng et al. (NeurIPS 2022); the Eiffel Tower example is on the project page, rome.baulab.info. The paper's own examples edit the Space Needle and Pierre Curie.

More facts than dimensions?

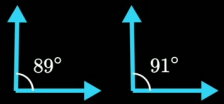


3Blue1Brown

768 numbers per token, yet a model knows far more than 768 things. How?

In many dimensions, directions do not have to be exactly perpendicular to stay out of each other's way: nearly perpendicular (89 to 91 degrees) is enough, and you can fit far more of those than there are dimensions.

This idea - many features squeezed into fewer dimensions - is called **superposition**. More in the interpretability chapter.

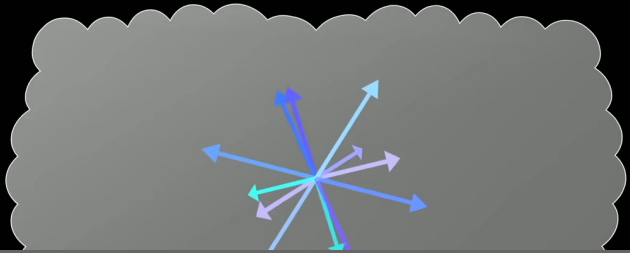


Choose multiple vectors,
each pair between 89° and 91° apart

Johnson–Lindenstrauss Lemma

$\Longrightarrow$ Maximum # of vectors: $\approx \exp(\epsilon \cdot N)$

N-dimensional Space



Johnson-Lindenstrauss: the number of nearly perpendicular directions grows exponentially with the dimension.

Turn one direction up: Golden Gate Claude

- Anthropic (May 2024) pulled millions of such directions, called **features**, out of the residual stream of Claude 3 Sonnet with a sparse autoencoder. One fired on the Golden Gate Bridge.
- They clamped that one direction at 10 times its usual maximum and asked "what is your physical form?". Claude: "I am the Golden Gate Bridge... my physical form is the iconic bridge itself."
- The usual answer: "I have no physical form, I am an AI model." Golden Gate Claude was online for 24 hours as a research demo.

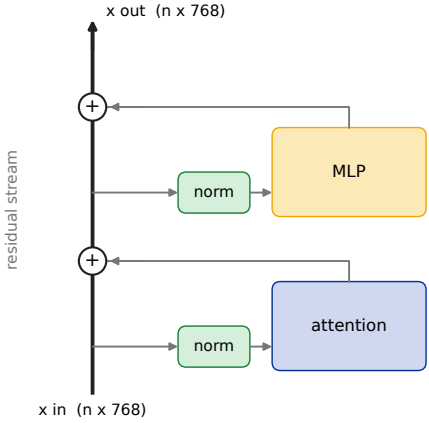
A concept is a direction, and behaviour follows what is written into the stream. Features, superposition and sparse autoencoders: the interpretability chapter.

Anthropic, "Golden Gate Claude" (23 May 2024); Templeton et al. (2024), "Scaling Monosemanticity", Transformer Circuits Thread.

The block

What you actually stack.

The Transformer block



Attention: tokens exchange information. The *only* place in the block where tokens see each other.

MLP: each token alone; two thirds of the weights.

Norm before each, and the result **added back**: LayerNorm from [44], the residual connection from [49].

Every piece but attention is something you already met. GPT-2's order: norm, sublayer, add.

The residual stream is the real object

Read the block as a **stream** of token vectors running straight through the network, and each sublayer *writing an update into it*:

$$\mathbf{x} \leftarrow \mathbf{x} + \mathrm{Attention}\big(\mathrm{LN}(\mathbf{x})\big) \qquad \text{then} \qquad \mathbf{x} \leftarrow \mathbf{x} + \mathrm{MLP}\big(\mathrm{LN}(\mathbf{x})\big) \qquad (\mathrm{LN} = \text{LayerNorm})$$

Unroll it over all L sublayers, with $\mathbf{u}_i$ the update sublayer i writes:

$$\mathbf{x}_L = \mathbf{x}_0 + \mathbf{u}_1 + \mathbf{u}_2 + \cdots + \mathbf{u}_L$$

The stream itself is never normalised (only once, at the very top, before the LM head: LLM-5). Each sublayer reads a normalised copy and adds a **correction**, so every update is still in the sum, untouched, and the gradient reaches each $\mathbf{u}_i$ directly: the derivative of a sum - the ResNet argument from [49]. Normalise the stream after every add instead, and each norm rescales everything written so far.

GPT-2 moved the norm "to the input of each sub-block" (Radford et al., 2019). The 2017 paper normalised after the add: $\mathbf{x} \leftarrow \mathrm{LN}(\mathbf{x} + f(\mathbf{x}))$.

By hand: LayerNorm on creature

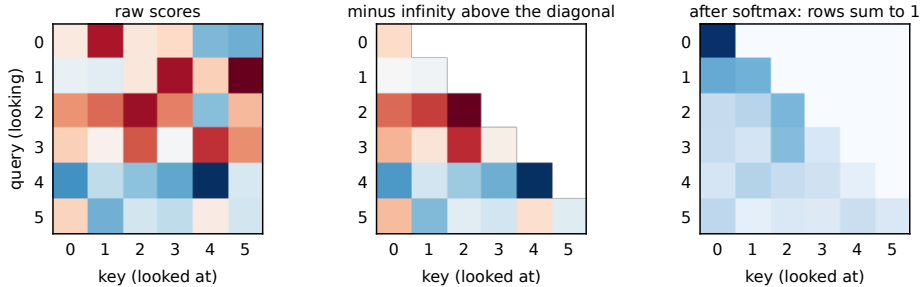
LayerNorm ([44]) rescales one token's vector so its numbers have mean 0 and spread 1. Creature after LLM-3, $(0,\ 0.47,\ 0.47,\ 2.13)$:

mean	$(0 + 0.47 + 0.47 + 2.13)/4 = 0.77$
spread	$\sqrt{\frac{1}{4}\sum(x_i - 0.77)^2} = 0.81$
normalised	$(x - 0.77)/0.81 = (-0.95,\ -0.37,\ -0.37,\ 1.68)$
then	times a learned scale, plus a learned shift (1 and 0 here)

Each token is normalised on its own numbers - not across the batch, as BatchNorm does ([44]). Attention and the MLP read this copy, $(-0.95, -0.37, -0.37, 1.68)$; the stream keeps $(0, 0.47, 0.47, 2.13)$ and gets their update added.

Masking: a decoder must not read the future

A model trained to predict the next token must not be allowed to look at it. The fix is one line: add $-\infty$ above the diagonal *before* the softmax.



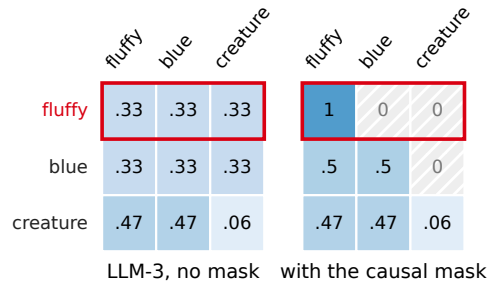
$e^{-\infty} = 0$, so those weights become **exactly** zero, and each row still sums to one.

By hand: the mask on creature's grid

LLM-3's grid (queries read "noun", keys read "adjective"), now with the mask:

- **fluffy** (red) sees only itself: $(0.33, 0.33, 0.33) \to (1, 0, 0)$;
- blue sees fluffy and itself, scores $0, 0$: $(0.5, 0.5, 0)$;
- creature's row does not change: $(0.47, 0.47, 0.06)$.

Creature's adjectives came first, so the mask took nothing from it.



LLM-3, no mask with the causal mask

In English adjectives come before their noun, so a left-to-right model can still let the noun gather them. What the mask removes is the other direction.

Why that trick matters more than it looks

Without the mask, position t could look at token $t+1$ - the very answer it must predict. So each prediction would need its own pass with the future cut off: feed the first t tokens, predict token $t+1$, repeat.

With masking, one forward pass over n tokens gives n **training signals at once**: every prefix is predicted at the same time, and none of them can cheat.

A large part of why pretraining on internet-scale text is affordable at all. The causal mask is not a safety measure; it is a **throughput** decision that happens to enforce correctness. (LLM-5 and LLM-7 come back to it.)

Modern block recipe

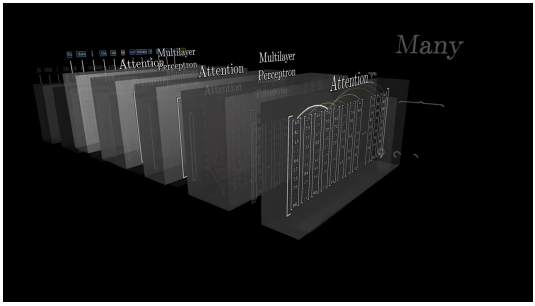
The block has been tuned since 2017. Four changes you will meet in current models:

2017	Now	Why
norm after the add	norm first (GPT-2 already)	trains more stably; can even skip warm-up
LayerNorm	**RMSNorm**	cheaper: no mean, only the spread
ReLU in the MLP	**SwiGLU**	a gated MLP; better quality in Shazeer's tests
MLP 4× wide	2.7× wide, 3 matrices	about the same weights

Xiong et al., ICML 2020 (norm first); Zhang & Sennrich, NeurIPS 2019 (RMSNorm); Shazeer, 2020 (GLU variants). Width: Llama 2 7B goes 4,096 → 11,008 (config.json); $3 \times 2.7 \approx 2 \times 4$.

None of these changes the story: attention, an MLP, norms and the residual stream. LLM-14 opens a 2026 model card and finds the same block.

Now stack it



3Blue1Brown

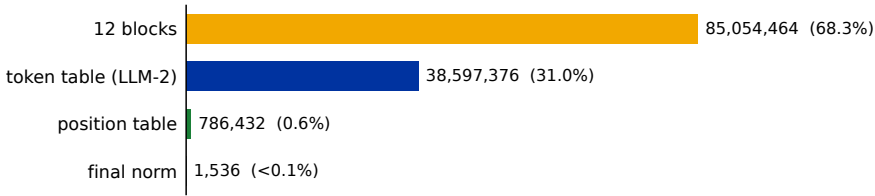
A block takes $n \times 768$ numbers in and gives $n \times 768$ out - the same shape, which is why blocks can be stacked. GPT-2 small stacks **12**.

Each block alternates **tokens talking** (attention) with **each token thinking alone** (the MLP).

A token's vector starts as "the word *creature*" and, layer by layer, gathers what the context implies: fluffy, blue, cute, and whatever comes next.

Same block, 12 times. That is the whole architecture. And since every layer only *adds*, you can read the stream after any layer through the final LM head and watch the guess form: the "logit lens" (2020).

Where GPT-2's 124M live



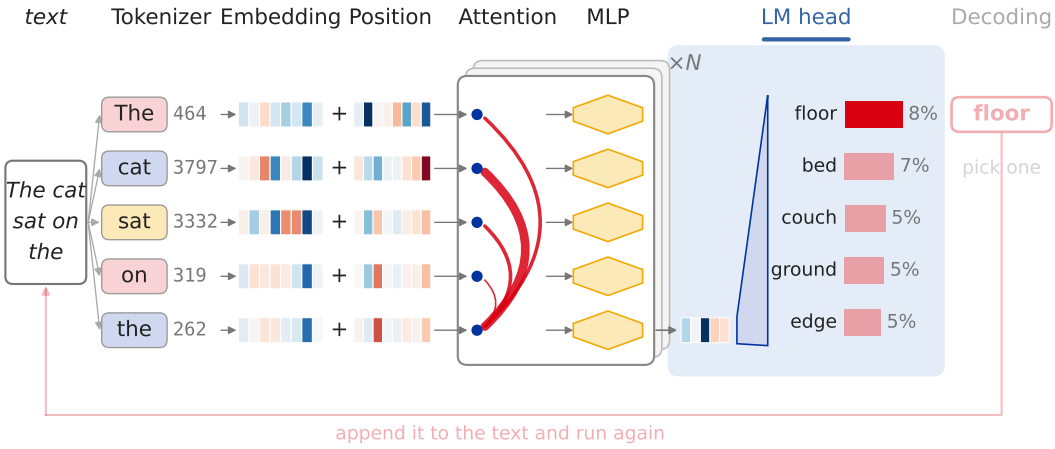
$12 \times 7{,}087{,}872 = 85{,}054{,}464$ in the blocks; the token table (LLM-2's 31%); 1,024 learned positions; one final norm. Total **124,439,808**, read from the weight file.

Two thirds of every block is MLP, and the blocks are two thirds of the model. Attention, the star of LLM-3, is under a quarter of GPT-2 (22.8%).

Recap

- **Many heads**: h questions at once, each in a smaller space (GPT-2: $12 \times 64 = 768$), mixed by W_O. Real heads: one looks one token back, one sends creature to fluffy, many park on the first token.
- **Heads team up**: a previous-token head plus an induction head copy $\ldots$ A B $\ldots$ A $\rightarrow$ B from the prompt. Sinks went from quirk to design in 2025.
- **Still blind to order**: one head or twelve, so positions come in before the first layer (LLM-2).
- **The MLP**: rows ask, ReLU gates, columns add. A neuron can fire on "fluffy AND creature", and facts like Messi's sport seem to live here: edit one MLP and the Eiffel Tower moves to Rome. Two thirds of every block.
- **The block**: norm, attention, add; norm, MLP, add. The stream is never normalised; every sublayer writes a correction.
- **The mask**: no reading the future, and n training signals per pass.

Next on the map



Next (LLM-5): what comes out at the far end, and three ways to wire the same block.

Sources & further reading

- **Watch:** 3Blue1Brown (2024), How might LLMs store facts | Deep Learning Chapter 7; Karpathy (2023), Let's build GPT: from scratch, in code, spelled out. **Play:** Transformer Explainer.
- Vaswani et al. (2017), "Attention Is All You Need", NeurIPS; Radford et al. (2019), "Language Models are Unsupervised Multitask Learners" (GPT-2).
- Olsson et al. (2022), "In-context Learning and Induction Heads", Transformer Circuits Thread.
- Xiao et al. (2024), "Efficient Streaming Language Models with Attention Sinks", ICLR; Miller (2023), "Attention Is Off By One" (blog); OpenAI (2025), gpt-oss model card; Qiu et al. (2025), "Gated Attention for Large Language Models", NeurIPS.
- Geva et al. (2021), "Transformer Feed-Forward Layers Are Key-Value Memories", EMNLP; Meng et al. (2022), "Locating and Editing Factual Associations in GPT", NeurIPS; Kandpal et al. (2023), "Large Language Models Struggle to Learn Long-Tail Knowledge", ICML.
- Templeton et al. (2024), "Scaling Monosemanticity", Transformer Circuits Thread; nostalgebraist (2020), "interpreting GPT: the logit lens", LessWrong.
- Xiong et al. (2020), "On Layer Normalization in the Transformer Architecture", ICML; Zhang & Sennrich (2019), "Root Mean Square Layer Normalization", NeurIPS; Shazeer (2020), "GLU Variants Improve Transformer", arXiv.